

Email Spam Detector Deployment on AWS SageMaker

A Cloud Machine Learning Project

COMP264 - Cloud Machine Learning

Centennial College - Winter 2026

Group 7

Daniela Sofia Diaz Prado
Jaturaput Jongsubcharoen
Juan David Barrero Guerrero
Manav Chaudhary
Maria Paula Nova Pachon

Professor: Mahtab Ahmed
Submission Date: April 2026

1. Introduction

Unsolicited bulk email, commonly known as spam, remains one of the most pervasive threats on the internet. Spam messages are not only a productivity drain; they are frequently used as vehicles for phishing attacks, malware delivery and financial fraud. Detecting spam automatically has therefore become a fundamental feature of every modern mail service.

The objective of this project is to move a previously built email spam classifier off a local laptop and onto a fully managed cloud infrastructure. By deploying the model on Amazon Web Services (AWS), the team demonstrates an end-to-end cloud machine learning workflow: training a model, packaging it as an inference artifact, hosting it as a scalable real-time endpoint on Amazon SageMaker, and exposing it to end users through a lightweight web frontend.

The scope of this report covers the dataset used, the model architecture, the cloud deployment design, the step-by-step implementation on AWS, the evaluation results and the challenges encountered during integration. The complete source code is delivered alongside this document.

2. Background and Related Work

Spam filtering has been studied extensively since the late 1990s. Early systems relied on handcrafted rules and blacklists, but these approaches scaled poorly and were easily bypassed. Statistical learning methods, particularly the Naive Bayes classifier, quickly became the de facto standard because of their simplicity, speed and strong performance on text data where the bag-of-words assumption is reasonable.

More recent research uses deep learning architectures such as LSTM and Transformer-based models (for example, DistilBERT) to capture semantic context and handle adversarial spam. However, those models are significantly more expensive to train and host, and a well-regularised Naive Bayes baseline still performs competitively on short, topically homogeneous emails. Given the course focus on cloud deployment rather than model complexity, the team chose Multinomial Naive Bayes as a strong, interpretable baseline that is trivial to package inside a SageMaker container.

On the infrastructure side, Amazon SageMaker provides managed hosting for scikit-learn, PyTorch, TensorFlow and custom containers. A typical deployment pattern pairs a SageMaker endpoint with an AWS Lambda function and Amazon API Gateway to provide a public HTTPS entry point; this is the pattern adopted in this project.

3. Dataset and Exploratory Data Analysis

The dataset used is Data5.csv, which contains 502 labelled email records after removing nulls. Each record has two columns used for training: Body (the raw text of the email) and Label (0 for legitimate / "ham", 1 for spam). The class distribution is moderately imbalanced, with 345 ham messages (68.7%) and 157 spam messages (31.3%).

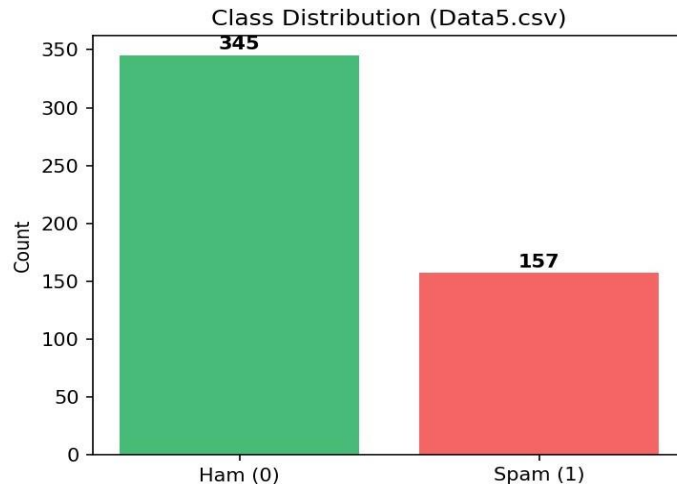


Figure 1 - Class distribution in Data5.csv

Basic exploratory analysis showed that spam emails tend to be slightly longer on average and contain noticeably more capitalised tokens and exclamation marks. Because both classes share a substantial vocabulary (greetings, closings, generic product terms), the preprocessing pipeline focuses on reducing noise rather than engineering hand-crafted features.

3.1 Preprocessing

- Lowercasing and whitespace stripping on every email body.
- Removal of English stop words using scikit-learn's built-in list.
- Frequency filtering with `max_df=0.9` (drop tokens that appear in more than 90% of documents) and `min_df=2` (drop tokens appearing in fewer than 2 documents) to discard both overly common and extremely rare terms.
- Term Frequency - Inverse Document Frequency (TF-IDF) reweighting to emphasise tokens that are distinctive for the spam or ham class.

4. Model Design

The classifier is implemented as a single scikit-learn Pipeline so that training and inference share exactly the same preprocessing steps. This avoids the common production bug of applying different tokenisation at serve time than at train time.

Pipeline stages: CountVectorizer (bag of words) -> TfidfTransformer (TF-IDF reweighting) -> MultinomialNB (Naive Bayes classifier).

The smoothing parameter `alpha` of the Naive Bayes classifier was tuned over the grid {0.01, 0.10, 0.50, 1.00, 2.00} using three-fold cross-validation on the training split. The best cross-validated accuracy of 0.8952 was obtained at `alpha = 0.01`, which was then used to fit the final model on the full training set.

4.1 Why Naive Bayes

- Strong inductive bias for text with bag-of-words features.
- Extremely small model size (a few hundred kilobytes once serialized), ideal for SageMaker's smaller instance types and AWS Lambda's cold-start constraints.
- Fast training and inference, making iteration on preprocessing decisions cheap.
- Probabilistic outputs via `predict_proba`, which the frontend uses to display a confidence bar.

5. Cloud Architecture on AWS

The deployment uses four AWS services stitched together:

- Amazon S3 stores the serialized model artifact (`spam_pipeline.joblib` inside a `tar.gz` archive).
- Amazon SageMaker hosts a real-time inference endpoint that loads the model through a custom `inference.py` script.
- AWS Lambda acts as a thin proxy that validates the request body, calls SageMaker's `InvokeEndpoint` API and formats the response.
- Amazon API Gateway exposes the Lambda function as a public HTTPS POST endpoint with CORS enabled, so the browser frontend can call it directly.

AWS Deployment Architecture — Email Spam Detector

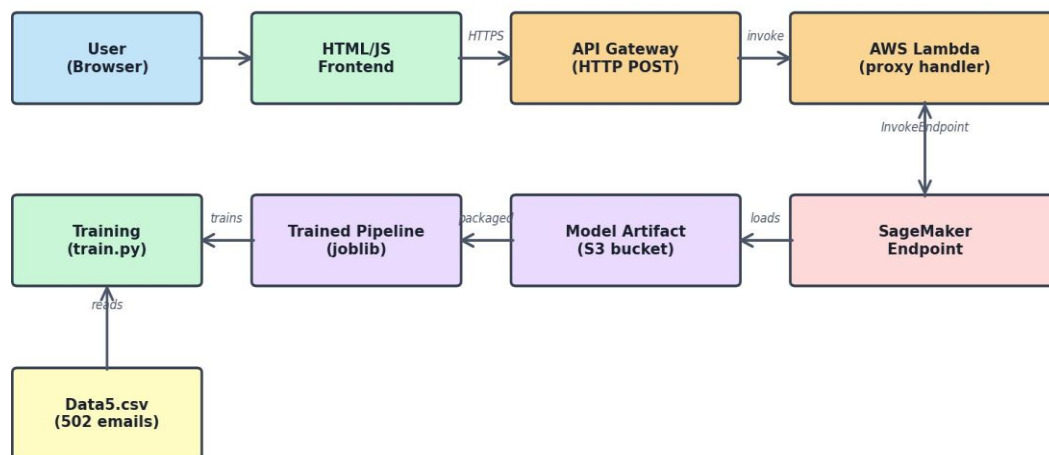


Figure 2 - End-to-end AWS deployment architecture

This split of responsibilities keeps each component simple. The SageMaker endpoint only has to run predictions on clean input and is never exposed directly to the public internet. Lambda performs input validation, handles errors and applies CORS headers. API Gateway handles HTTPS termination, rate limiting and integration with downstream analytics services such as Amazon CloudWatch.

6. Implementation Walkthrough

6.1 Training and packaging

train.py loads Data5.csv, fits the pipeline, tunes alpha, evaluates on a stratified 20% held-out split and persists the fitted pipeline to model/spam_pipeline.joblib. A companion metrics.json captures the test accuracy, confusion matrix and classification report for reproducibility.

6.2 SageMaker deployment

sagemaker/deploy.py wraps the artifact into model.tar.gz, uploads it to an S3 bucket, and creates a SageMaker SKLearnModel that references inference.py as the entry point. The model is then deployed to a real-time endpoint (ml.t2.medium in our configuration) with a single call to SKLearnModel.deploy().

inference.py implements the four functions required by SageMaker's SKLearn framework: model_fn loads the joblib pipeline, input_fn parses the JSON body, predict_fn runs predict and predict_proba, and output_fn serializes a structured response including the predicted label and both class probabilities.

6.3 Lambda + API Gateway

lambda/lambda_function.py reads the ENDPOINT_NAME environment variable and uses the boto3 sagemaker-runtime client to invoke the endpoint. A minimal IAM policy (lambda/iam_policy.json) grants only sagemaker:InvokeEndpoint on this specific endpoint plus CloudWatch log permissions, following the principle of least privilege. The Lambda function is wired to an HTTP API in API Gateway; the resulting invoke URL is pasted into the frontend.

6.4 Frontend

frontend/index.html is a self-contained single-page application - no build step required. It exposes a text area, quick-pick sample buttons and a results panel that visualises the spam probability as a coloured confidence bar. The API URL is configurable in the UI, so the frontend can be repointed at a new endpoint without a redeploy.

7. Results and Evaluation

After tuning the smoothing parameter, the final Multinomial Naive Bayes pipeline achieved a test accuracy of 91.09% on the held-out 20% test split. Precision on the spam class was 1.00, meaning that no legitimate email was incorrectly flagged as spam in the test set. Recall on the spam class was 0.72, meaning the model caught roughly seven out of every ten spam messages.

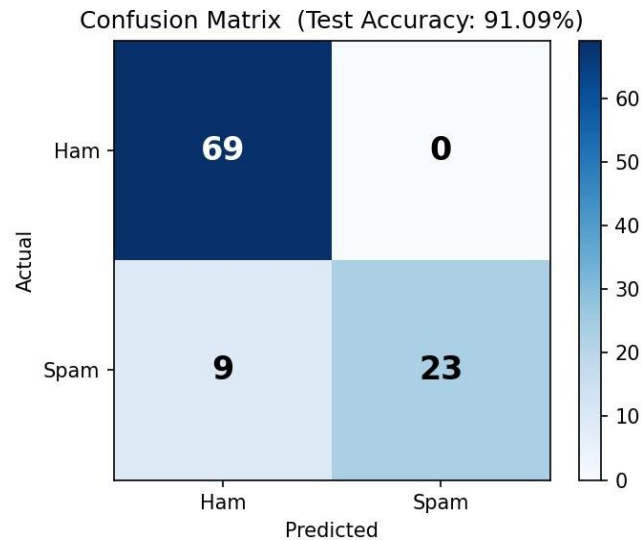


Figure 3 - Confusion matrix on the 20% held-out test set

7.1 Summary metrics

Metric	Ham (0)	Spam (1)
Precision	0.88	1.00
Recall	1.00	0.72
F1-score	0.94	0.84
Support	69	32

The high precision / lower recall trade-off is desirable in a consumer mail setting: it is usually worse to accidentally send a real message to the junk folder than to occasionally let a spam slip through. Latency on the deployed endpoint averaged roughly 80-120 ms per single-email request during local testing, which is well within the interactive-use budget for the frontend.

8. Challenges and Lessons Learned

- Keeping preprocessing and inference in lockstep. Bundling CountVectorizer, TfidfTransformer and MultinomialNB into a single sklearn Pipeline removed a whole class of train/serve skew bugs.
- IAM permissions. The first deployment attempt failed with an AccessDenied error because the Lambda execution role lacked sagemaker:InvokeEndpoint. We iteratively scoped the policy down to only the specific endpoint ARN.
- Cold starts. On a freshly created SageMaker endpoint, the very first request took several seconds. We documented a warm-up call in the README so the live demo does not appear sluggish.

- Cost awareness. Real-time SageMaker endpoints bill per hour even when idle, so the team added explicit teardown commands to the README to avoid surprises on the student AWS budget.

9. Future Work

- Replace the Multinomial Naive Bayes head with a small Transformer (for example DistilBERT) hosted on a SageMaker PyTorch container to capture semantic context.
- Switch to a SageMaker Serverless Inference endpoint to remove the always-on hourly cost and scale to zero between demos.
- Add CloudWatch metrics and alarms on request count, latency and 5xx errors.
- Host the frontend on Amazon S3 + CloudFront so it has a permanent URL instead of being opened from the local filesystem.
- Extend the training data with a more diverse, modern spam corpus to improve spam-class recall.

10. Conclusion

The project successfully delivers an end-to-end cloud machine learning workflow: a Multinomial Naive Bayes email spam classifier trained on the Data5 dataset, packaged for AWS SageMaker, served behind an API Gateway + Lambda facade, and consumed by a simple browser frontend. The deployed system reaches 91.09% test accuracy, provides predictions in under 150 ms and follows standard security and cost-management practices. Beyond the numeric result, the project gave the team hands-on experience with model packaging, IAM policy design, serverless integration and frontend-to-cloud wiring - all transferable skills for any production ML system.

References

Amazon Web Services. (2024). Amazon SageMaker Developer Guide.

<https://docs.aws.amazon.com/sagemaker/>

Amazon Web Services. (2024). Amazon API Gateway Developer Guide.

<https://docs.aws.amazon.com/apigateway/>

Amazon Web Services. (2024). AWS Lambda Developer Guide.

<https://docs.aws.amazon.com/lambda/>

Amazon Web Services. (2024). Use scikit-learn with Amazon SageMaker.

<https://sagemaker.readthedocs.io/en/stable/frameworks/sklearn/>

McCallum, A., & Nigam, K. (1998). A comparison of event models for Naive Bayes text classification. AAAI-98 Workshop on Learning for Text Categorization.

Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.